

Jobify — Complete Project Explanation Guide

This document is a full-length explanation of the Jobify project for viva and evaluation. It covers database, backend, frontend, and complete user flows in detail.

1. Project Overview

Jobify is a full-stack job tracking application where users can manage job applications, share interview experiences, build resumes, and upgrade to premium features.

2. Features

- Authentication (JWT) - Job CRUD - Community sharing - Resume builder - Payment integration - Profile system

3. Database Design

The database is centered around the user. All entities such as jobs, resumes, and payments are linked to users.

users → core entity
jobs → user activity
community_posts → derived from jobs
resumes → structured data
payments → subscription system

4. Backend Architecture

The backend follows MVC architecture with routes, controllers, models, and middleware.

Frontend → Route → Middleware → Controller → Model → DB → Response

5. Frontend Architecture

The frontend is built using React with component-based architecture and context for global state.

6. Add Job Full Flow

User fills form
→ Click Save
→ Validation
→ API call
→ Backend processing
→ DB insert
→ Response
→ UI update

7. Authentication Flow

Login → JWT → stored → used in headers → verified in middleware

8. Payment Flow

Create Order → Payment → Verify → Update DB → Upgrade user

9. Community Flow

Job shared → community post created

Edit → update
Unshare → delete

10. Resume Builder Flow

User inputs data → preview updates → save → export PDF

11. Design Decisions

The project uses JWT for stateless authentication, JSON for flexible resume data, and derived community posts for consistency.

12. Conclusion

The system demonstrates full-stack development with proper architecture, data flow, and user experience design.

Jobify — Deep Backend Explanation Guide

This document is designed for viva/exam preparation. It explains backend architecture, request lifecycle, middleware, and complete flows with reasoning.

1. Backend Architecture (MVC Pattern)

```
backend/  
■■■ routes/  
■■■ controllers/  
■■■ models/  
■■■ middleware/  
■■■ config/  
■■■ server.js
```

- 1 Routes: Define API endpoints
- 2 Controllers: Handle business logic
- 3 Models: Interact with database
- 4 Middleware: Authentication, validation, preprocessing

2. Request Lifecycle (Critical)

Frontend → Route → Middleware → Controller → Model → Database → Response → Frontend

Every user action follows this exact pipeline. Understanding this flow is essential for explaining the system.

3. Authentication Flow (JWT)

Login:

```
User submits credentials  
→ Backend verifies user  
→ JWT token created  
→ Token sent to frontend  
→ Stored in localStorage
```

Protected Request:

```
Frontend sends token in header  
→ Middleware verifies token  
→ req.user populated  
→ Controller executes
```

4. Middleware (Deep Understanding)

Middleware acts as a gatekeeper before reaching controllers. It is used for authentication, logging, and preprocessing.

```
const token = req.headers.authorization.split(" ")[1];  
const decoded = jwt.verify(token, SECRET);  
req.user = decoded;
```

- 1 Extract token
- 2 Verify authenticity
- 3 Attach user to request

5. Add Job Flow (Step-by-Step)

User clicks 'Add Job'

- POST /jobs
- authMiddleware validates user
- Controller receives form data
- Model inserts into jobs table
- If share = 1 → insert into community_posts
- Response sent

6. Edit Job Flow

User edits job

- PUT /jobs/:id
- Controller updates jobs table
- If shared → update community_posts
- Response returned

7. Payment Flow (Critical)

Create Order:

- Backend creates Razorpay order
- Insert payment record (status=created)

Verify Payment:

- Validate signature
- Update payment = paid
- Update user.subscription_type = premium

8. Community Access Flow

GET /community

- optionalAuth middleware
- subscriptionMiddleware
- If free → limit 5 posts
- If premium → full access

9. Error Handling Strategy

- 1 Try-catch blocks in controllers
- 2 Return appropriate HTTP status codes
- 3 Frontend displays toast messages

10. Design Decisions (Important for Viva)

- 1 MVC architecture for separation of concerns
- 2 JWT for stateless authentication
- 3 Derived community posts for consistency
- 4 Payment table for auditability

11. How to Explain in Interview

The backend follows a structured MVC architecture. Requests flow from frontend to routes, pass through middleware for authentication, are handled by controllers, interact with models for database operations, and return structured responses. This ensures maintainability, scalability, and security.

Jobify — Deep Frontend Explanation Guide

This document explains frontend architecture, state management, and full click-to-flow breakdown for viva preparation.

1. Frontend Architecture

```
src/  
■■■ pages/  
■■■ components/  
■■■ context/  
■■■ services/  
■■■ App.jsx
```

- 1 Pages: Main screens
- 2 Components: Reusable UI
- 3 Context: Global state (auth)
- 4 Services: API handling

2. Authentication Flow

```
Login click  
→ Form submit  
→ API call /auth/login  
→ Token + user received  
→ Stored in localStorage  
→ Context updated  
→ Redirect to dashboard
```

3. Protected Route Flow

```
Page loads  
→ AuthContext loads user  
→ If loading → wait  
→ If no user → redirect login  
→ Else → render page
```

4. Add Job Click Flow

```
User fills form  
→ Click Save  
→ Validate fields  
→ API POST /jobs  
→ Backend saves  
→ Success response  
→ Toast shown  
→ Redirect to jobs page
```

5. Edit Job Flow

```
User clicks edit  
→ Load job data  
→ Modify fields  
→ Click update  
→ API PUT /jobs/:id
```

- Backend updates
- Toast + redirect

6. Community Flow

- User opens community
- API GET /community
 - Backend returns limited/full posts
 - UI renders posts

7. Resume Builder Flow

- User types data
- State updates
 - Preview updates live
 - Save → POST /resume
 - Download → generate PDF

8. Navbar Flow

- User login
- Context updated
 - Navbar shows name + badge
 - Click profile → navigate /profile

9. Complete Click Flow

- User action
- Event handler
 - State update
 - API call
 - Backend processing
 - Response
 - UI update

10. Interview Explanation

The frontend is built using React with component-based architecture. State is managed using hooks and context. User actions trigger API calls, and responses update the UI dynamically, ensuring a responsive and interactive experience.